

Ingegneria del Software – Parte II

Sviluppo Agile: XP



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Prof. Alessandro Saetti

Per la partecipazione interattiva...



Cosa significa agile?

- Sta ad indicare metodologie per gestire team e progetti di IT
- Deriva dal manifesto agile <http://agilemanifesto.org>
- Pubblicato nel 2001 da 17 firmatari, tra cui Kent Beck, Robert C. Martin, Martin Fowler



3

Manifesto for Agile Software Development*

"We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

 Individuals & interactions	 Processes & tools	 Working software	 Comprehensive documentation
 Customer collaboration	 Contract negotiation	 Responding to change	 Following a plan

That is, while there is value in the items on the right, we value the items on the left more."

Knowledge TRAIN

Copyright © 2016 Knowledge Train Limited. www.knowledgetrain.co.uk. *Quoted from www.agilemanifesto.org

4

The 12 agile principles*

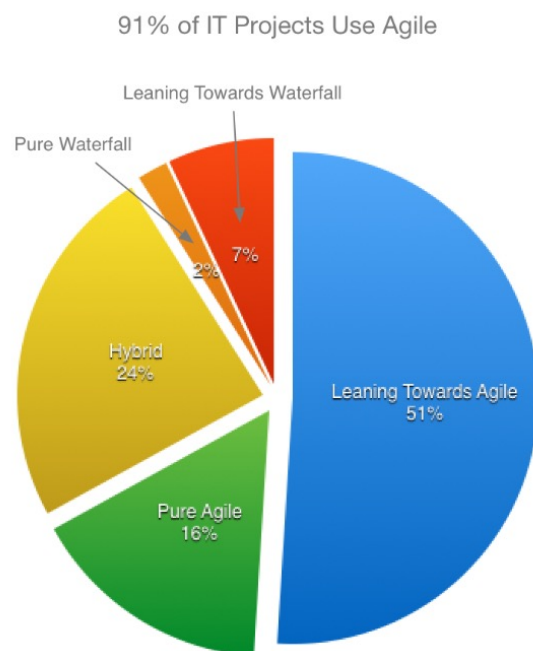
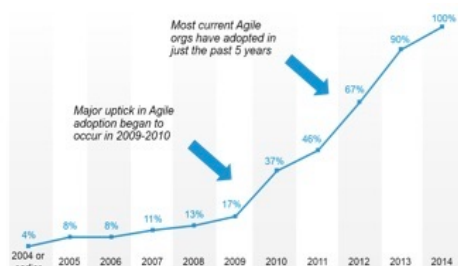


5

Agile negli ultimi anni e tutt'oggi

Utili quando:

- Requisiti cambiano frequentemente
- Società/prodotti di piccole/medie dimensioni



Primary development method used in organization across projects.
Source: HP online survey of 601 development and IT professionals.
<http://techbeacon.com/survey-agile-new-norm>

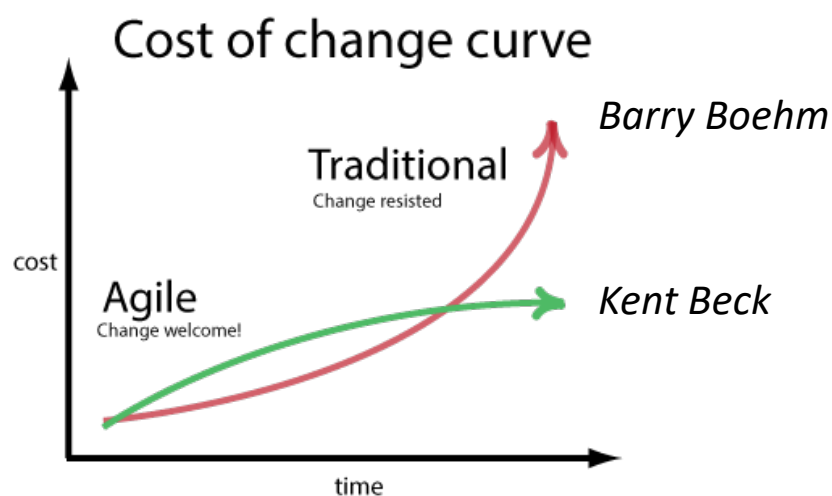
6

Agile o pianificato?

Fattore	Agile	Pianificato
Dimensione	Adatto a prodotti piccoli	Adatto a prodotti grandi
Criticità	Utile se requisiti instabili	Utile se requisiti stabili o per applicazioni critiche
Cambiamento	Refactoring	Pianificazione dettagliata
Esperti	Nei metodi agili	Nella pianificazione
Aspetti culturali	Piace a chi preferisce avere libertà di fare	Piace a chi preferisce ruoli e procedure ben definiti

7

La vera differenza!



8

Modellare in Agile: storie utente

Descrive in modo semplice una funzionalità o un requisito che è di interesse per il cliente

- Non contengono dettagli sui requisiti
- Dettagli discussi a voce coi clienti

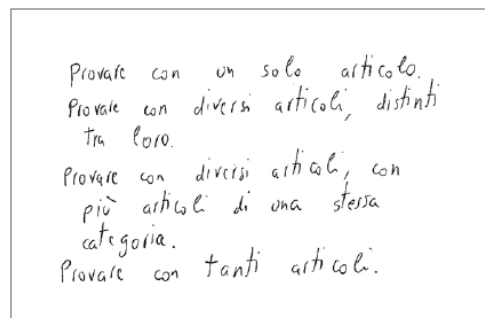
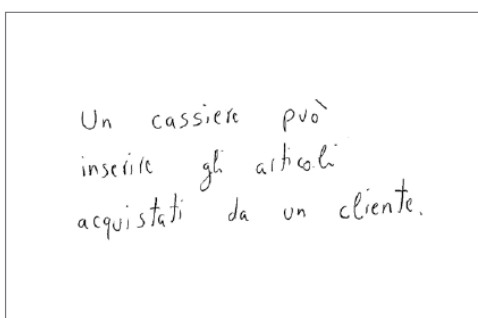


9

Esempi di storia utente

Scritta su una piccola scheda cartacea (13 x 20 cm)

- Descrizione della storia
- Test di accettazione (esito della conversazione)



➤ Promemoria utili durante la pianificazione

Discussione

- Le storie utenti sono usate per pianificare l'iterazione
- Quanto sono lunghe queste storie?
 1. Un cassiere può gestire la vendita per un cliente
 2. Un cassiere può inserire gli articoli acquistati da un cliente
 3. La ricevuta di una vendita riporta l'ora
- Quanto dovrebbe durare una storia?



11

Discussione



12

Storie troppo grandi o piccole

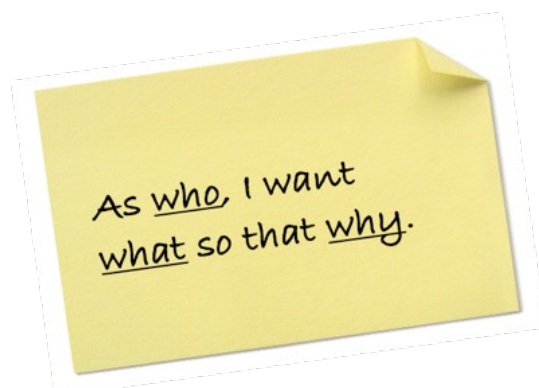
- **Epoepa:** storia composta da suddividere
 - Se la storia riguarda un intero caso d'uso si decompone in
 - Una storia per lo scenario principale (da decomporre ulteriormente, se necessario)
 - Più altre storie per scenari secondari
- **Storia complessa:** storia intrinsecamente complessa non facilmente suddividibile
 - Se la storia è complessa a causa di nuove tecnologie si decompone in
 - Una storia esplorativa (spike)
 - Una storia per l'implementazione della nuova caratteristica
- Storie troppo piccole sono combinate assieme

13

Linee guida per le storie

Chi, che cosa, e perché!

- L'utente che ha interesse per la storia
- L'obiettivo e le funzionalità di interesse
- Il valore di business associato alla storia



14

Punti storia



15

Anti-metodologico?

- The Agile movement is not anti-methodology, in fact, many of us want to restore credibility to the word methodology. We want to restore a balance.
- We embrace modeling, but not in order to file some diagram in a dusty corporate repository.
- We embrace documentation, but not hundreds of pages of never-maintained and rarely-used tomes.
- We plan, but recognize the limits of planning in a turbulent environment.

Jim Highsmith, for the Agile Alliance

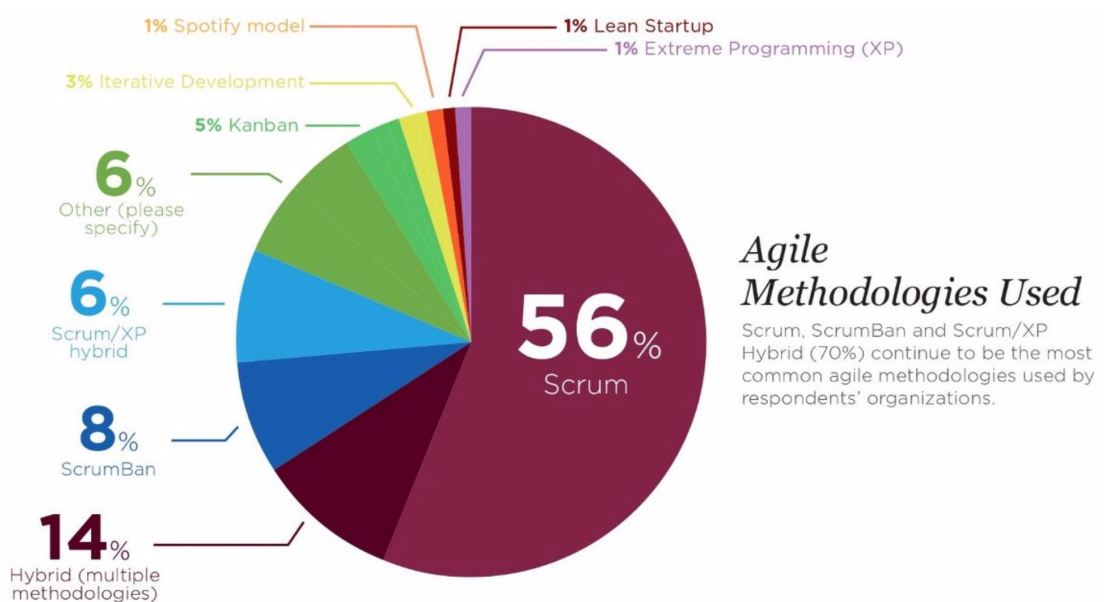
16

UML per agile modeling

- UML per agevolare comprensione e comunicazione
- UML per le parti più insidiose
- UML abbozzata alle lavagne
- UML in gruppo (2 o 3 persone)
- Modelli UML creati in parallelo
- La precisione nella notazione UML non importa
- UML come semplici esplorazioni usa e getta
- UML a carico degli sviluppatori stessi

17

Metodi agili



Agile Techniques Employed

From 2016 to 2017, the use of Kanban grew from 50% to 65%; product roadmapping increased from 38% to 46% and portfolio planning went from 25% to 35%.

18

UP può essere agile?

- I requisiti emergono dai feedback durante la programmazione
- Non esiste un piano dettagliato per l'intero progetto
- Si usano pochi elaborati
- Si usa UML con pratiche di agile modeling

19

Extreme Programming (XP)

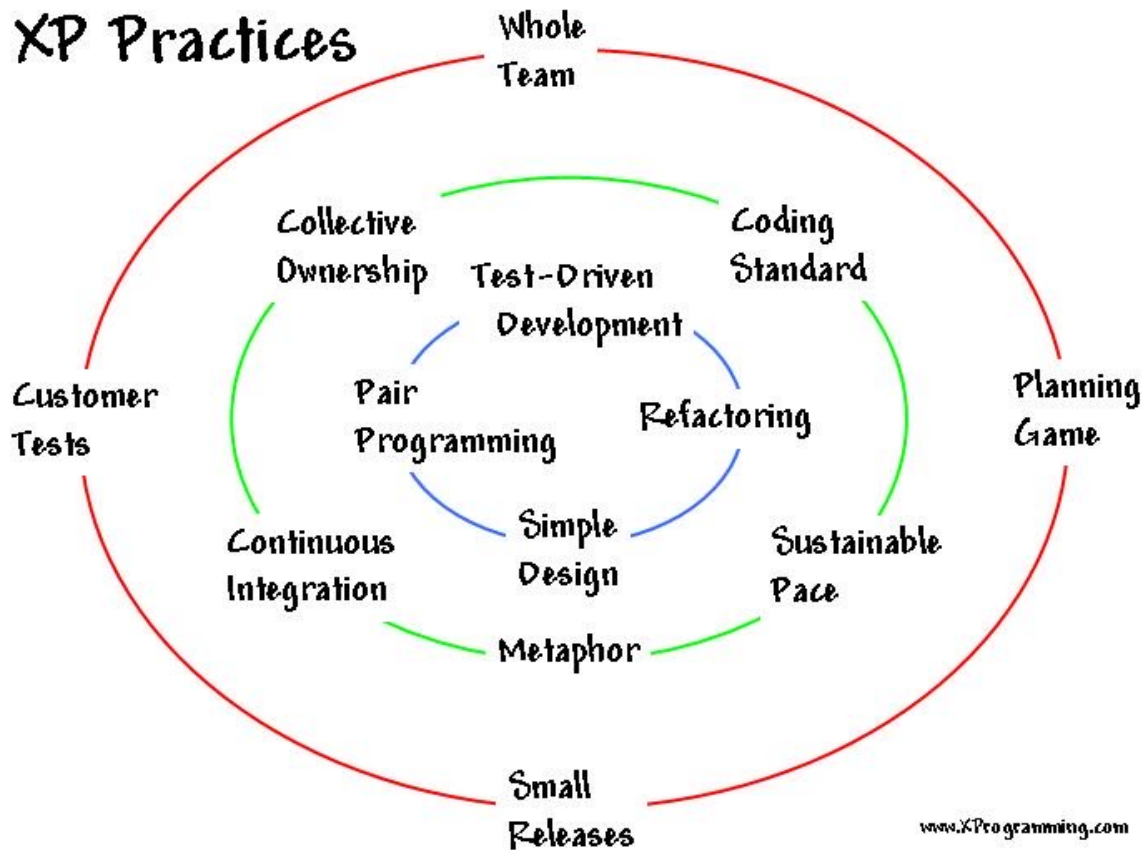
Extreme Programming (XP) is “a humanistic discipline of software development, based on values of communication, simplicity, feedback and courage”



People: Kent Beck, Ward Cunningham, Ron Jeffries, Martin Fowler, Erich Gamma...

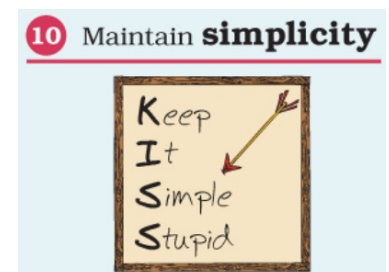
20

XP Practices



21

Progetta in modo semplice Cosa significa simple design per Beck?



Codice che passa i test e soddisfa i 4 requisiti (in ordine di importanza):

1. Appropriate for the intended audience
2. Communicative
3. Factored
4. Minimal (within the above three constraints)

Semplice non significa semplicistico!

Da Extreme Programming Explained (2nd edizione)

22

Progettare nella maniera più semplice

- Come fare? -

- **You Aren't Gonna Need It! (YAGNI)**: ciò che si progetta dovrebbe supportare le storie utente
- Rimuovere codice non più in uso
- **Once and only once!**
- Scrivere codice self-commenting
- Se il codice diventa troppo complesso, riscriverlo
- Vantaggi:
 - Minore investimento iniziale
 - Migliora la manutenibilità
 - Più facile/veloce cambiare rispetto a progetti complessi

23

Discussione

"The key to maximizing reuse lies in anticipating new requirements and changes to existing requirements, and in designing your systems so they can evolve accordingly. To design a system so that it's robust to such changes, you must consider how the system might need to change over its lifetime. A design that doesn't take change into account risks major redesign in the future."

The GoF book [Design Patterns]

Ma non è in contraddizione con la filosofia di XP?



24

Discussione



25

Refactoring

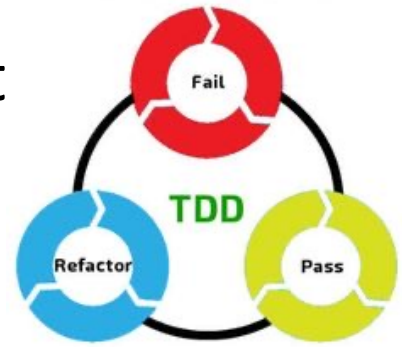
- Migliora il codice esistente senza modificarne il comportamento
 - Ridenominare o spostare cose
 - Scambiare logica procedurale con polimorfismo
 - Applicare design patterns
- Il refactoring combatte l'entropia
 - Si deve fare refactoring anche quando non c'è una immediata necessità
- Supportato da strumenti appositi degli IDE
- I test sono una rete di sicurezza durante il refactoring

26

Test driven development

- Come procedere -

I test sono scritti prima che il task sia implementato!



1. Scegliere un task di una storia utente
2. Coinvolgimento del cliente nella definizione dei test (di accettazione): come dei contratti!
3. Scrivere i test prima del codice
4. Si esegue il test insieme con tutti gli altri test
5. Si implementa il task e si ripete il test
6. Quando tutti i test sono superati si passa al task successivo o si rifattorizza all'occorrenza

27

Test driven development (TDD)

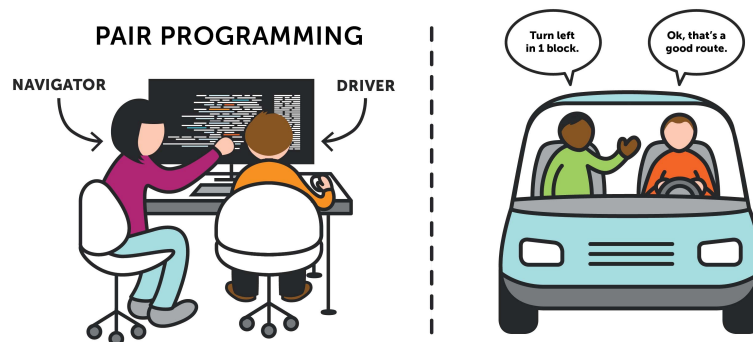
- Vantaggi -

- I problemi con la specifica dei requisiti si riducono
- Codice completamente testato
- Difetti sono scoperti nelle prime fasi dello sviluppo
- Possibile controllare che le modifiche non introducono bug
- Facile identificare dove è il bug
- I test documentano

28

Pair programming

- Due sviluppatori lavorano alla stessa tastiera aiutandosi vicendevolmente
- Il **driver** usa la tastiera e il mouse e codifica
- Il **navigator** osserva e pensa a
 - Ciò che il driver digita
 - Prossimi task da sviluppare
 - Come migliorare il codice esistente



29

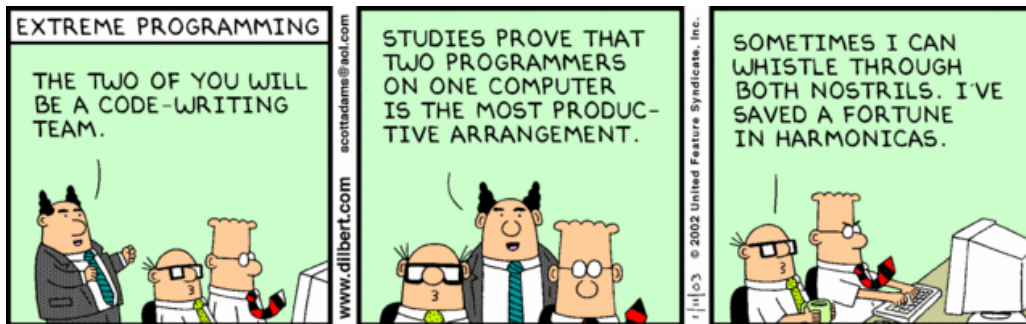
Pair programming - Fare le coppie -

- Usare la programmazione di coppia per l'intero codice
- Fare coppia progressivamente con tutti
- Le coppie si formano e sciolgono da sole
- Cambiare partner quando ci si sente bloccati lasciando uno sviluppatore sul task
- Cambiare partner anche più volte al giorno
- Sedie fianco a fianco e monitor visibile a entrambi
- I programmatori producono codice conversando:
pensare ad alta voce

30

Pair programming

- Vantaggi -



- 15% di sforzo in più contro più velocità e 15% di difetto in meno
- Migliora le pratiche di buona programmazione
- Si viene interrotti meno frequentemente
- Si è meno distratti dal brusio

31

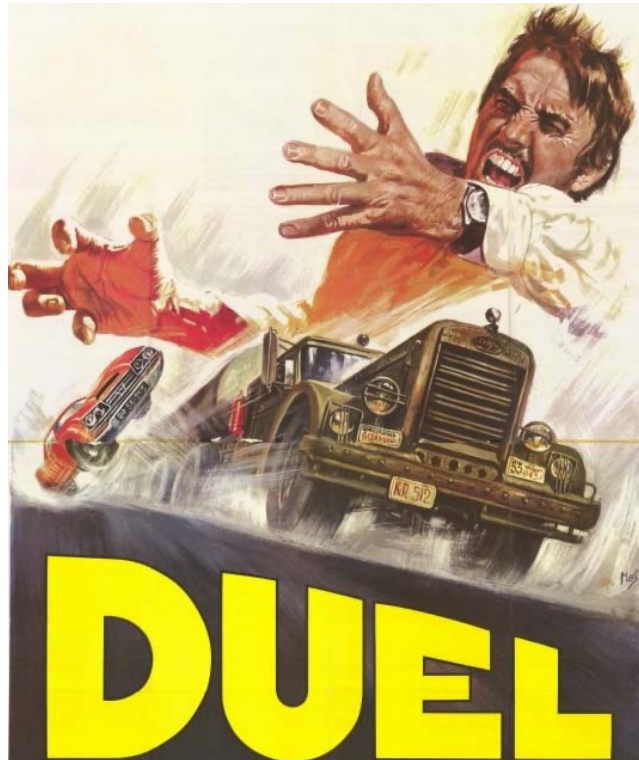
Integrazione continua

- Mantiene il codice di tutti integrato e costruisce l'infrastruttura di rilascio insieme al sistema
- L'obiettivo è potere rilasciare tutto (eccetto le ultime ore di lavoro) in qualsiasi momento
- Come integrare?
 - Aggiornare con la versione nel repository
 - Assicurarsi che tutto funzioni (test passati)
 - Caricare il proprio codice nella repository
- Quanto spesso integrare?
 - Cicli brevi rendono l'integrazione meno dolorosa

32

Proprietà collettiva

- La metrica del truck number -

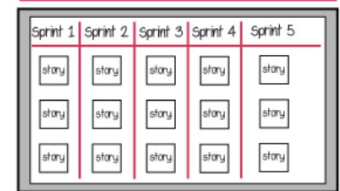


33

Proprietà collettiva

- Tutti condividono la responsabilità per la qualità del codice
- Tutti lavorano su tutto
 - Quando si lavora su un codice sconosciuto chiedere a chi lo ha scritto di fare coppia
 - Aiutare chi viene dopo rifattorizzando
- Liberarsi un po' del proprio ego!

34



Piccoli rilasci

- Ogni 3-4 mesi al massimo, fino ad ogni poche ore
- Incrementi minimali, ma utili
 - Mai cose come “implementare l’interfaccia utente”
- Vantaggi
 - Aumenta il feedback
 - Migliora la qualità del prodotto: il sistema deve essere sufficientemente buono per essere consegnato
- Dev-Ops spinge questa pratica all’estremo!

35



La squadra

- Intera squadra riunita
- I membri siedono alle postazioni di lavoro in un open space
 - We must always help when asked!
 - Tempi di rilascio ridotti a 1/3
- Almeno un cliente è sempre disponibile per chiarire storie e prendere decisioni critiche
 - Sono documentazione vivente
 - Un buon rapporto è 2 clienti ogni 3 programmatori
 - Se il cliente non viene al team, il team va al cliente!

36

Altre pratiche di XP

- **Planning game:** pianificazione approssimativa dell'iterazione da raffinare nel corso dell'iterazione
- **Metafora:** per sviluppare un linguaggio comune
- **Passo sostenibile:** «freschi e motivati ogni mattina, stanchi e soddisfatti ogni sera» dice Kent Beck
- **Codifica standard:** è più facile addentrarsi nel codice altrui se il team utilizza uno stesso stile di scrittura

37

La 13° pratica - La riunione in piedi -



- Ogni giorno a un orario prestabilito
- Per dare un'idea su a che punto siamo
- Siate concisi: 30 secondi a testa è sufficiente
- Uno alla volta, ognuno parla (da Scrum)
 - Cosa ho fatto ieri?
 - Cosa farò oggi?
 - Quali ostacoli sto incontrando?
- Se si ha bisogno non aspettare il meeting

38

Difetti di XP

- Carenza di documentazione
- Non promuove il design del software
- Il cliente non è disponibile ad essere coinvolto
- Non misura come sta andando il processo di sviluppo
- Funziona solo con sviluppatori molto esperti
- Richiede troppi cambiamenti culturali
- Difficile esprimere i requisiti non funzionali come storie
- C'è il rischio di una instabilità nello scopo del progetto a seguito di carenza di documentazione
- Può portare a negoziati difficili col cliente
- Difficile fare preventivi

39

Domande?



40

Il cambio di prospettiva

